

Decision Procedure for A Theory of String Sequences

Abstract. The theory of sequences, supported by many SMT solvers, can model program data types including bounded arrays and lists. Sequences are parameterized by the element data type and provide operations such as accessing elements, concatenation, forming sub-sequences and updating elements. Strings and sequences are intimately related; many operations, e.g., matching a string according to a regular expression, splitting strings, or joining strings in a sequence, are frequently used in string-manipulating programs. Nevertheless, these operations are typically not directly supported by existing SMT solvers, which instead only consider the generic theory of sequences. In this paper, we propose a theory of string sequences and study its satisfiability. We show that, while it is undecidable in general, the decidability can be recovered by restricting to the straight-line fragment. This is shown by encoding each string sequence as a string, and each string sequence operation as a corresponding string operation. We provide pre-image computation for the resulting string operations with respect to automata, effectively casting it into the generic OSTRICH string constraint solving framework. We implement the new decision procedure as a tool `OSTRICHSEQ`, and carry out experiments on benchmark constraints generated from both real-world and hand-crafted JavaScript programs. The experiments confirm the efficacy of our approach.

1 Introduction

Many real-world applications, such as web applications and programs processing data of type string, involve a multitude of complex operations that convert between strings and string sequences. Typical examples include `join` and `split`, which are present in almost all built-in libraries of modern programming languages. Reasoning about string sequences, also known as extendable string arrays [32], is therefore a crucial aspect of program analysis.

Satisfiability Modulo Theory (SMT [9]) solving provides an automatic way to verify software systems. At the moment, there is no standardized sequence theory in SMT-LIB [8]. Instead, in verification tools, often the theory of arrays is used, which is limited and does not provide even basic operations like sequence length. Fortunately, [14] proposed a basic format for a sequence theory, which has been implemented and extended by SMT solvers such as Z3 [25] and `cvc5` [7] with practical operations. In particular, [42] presents a calculus in `cvc5` for reasoning about string sequences, which extends the proposal [14] with `update` (aka `write`) operations. The implementation of sequences in Z3 has been documented in the Z3 guide [48] and includes operations like `map` and `foldleft` (but no `update`).

Although existing SMT solvers are effective at reasoning about sequences in general, to the best of our knowledge, none of them directly support functions

that convert between strings and string sequences such as `join` and `split` functions. While it is possible to implement these functions using existing ones for some SMT solvers (e.g., Z3 can use `map` and `foldleft` to implement the `join` function), usually the solvers do not provide decision procedures for those theories and fail to solve many instances. This limitation poses a significant challenge for program analysis that involves such operations. The purpose of our work is to fill this gap.

Contributions. We propose a logic (aka constraint language) **SeqStr** of string sequences, which extends the existing sequence theory in SMT solvers with dedicated string operations. In addition to standard string operations (`replace/replaceAll`, `reverse`, finite transducers, regular constraints, `length`, `indexOf`, `substring`, etc.), our logic emphasizes the interaction between strings and string sequences through operations such as `split`, `join`, `filter`, and `matchAll`. For instance, `split` splits a string into a sequence of substrings according to a regular expression; `join` concatenates all the strings in a sequence to obtain a single string. More details can be found in Section 4. Typically, when string sequences are considered, it is natural to consider the integer data type as well; for instance, one usually needs to refer to the length of a sequence or the index of specific sequence elements. As a result, **SeqStr** features three data types: integers, strings, and string sequences. In terms of operations, the logic subsumes, to our best knowledge, most string constraint languages that have been proposed in the literature.

Our main focus is the decision procedure for the satisfiability problem of **SeqStr**. Because of its expressiveness, it is perhaps not surprising that **SeqStr** is undecidable in general. To reinstate decidability, we consider the straight-line fragment, which imposes syntactic restrictions on the constraints written in the logic. Straight-line formulas naturally arise when verifying programs using bounded model checking or symbolic execution, which unroll loops in the programs up to a given depth and convert programs to static single assignment form (i.e. each variable is defined at most once). For example, a majority of the constraints in the standard Kaluza benchmarks [39] satisfy this condition. (Cf. Section 2 for a concrete example.)

The general strategy of our decision procedure is to encode each string sequence as a string in such a way that all operations for string sequences considered in **SeqStr** can be transformed into string operations, possibly involving integers. As such, the decidability of (straight-line) **SeqStr** is reduced to string constraints with the integer data type, which was shown to be decidable [19]. Needless to say, this strategy requires to overcome certain technical challenges, as the string constraints resulted from the reduction typically encompass complex (and sometimes non-standard) string operations, which go well beyond the capacity of state-of-the-art string constraint solvers. Recall that for the decidability of string constraints with integers, cost-enriched finite automata (CEFA), a variant of cost-register automata, were utilized [19]. The crux of our technical contributions is thus to compute the backward images of CEFA under the fairly complex string operations (cf. Section 5.2), so the powerful string constraint solving framework OSTRICH [17,20] can be harnessed.

We implement the decision procedure as a new solver $\text{OSTRICH}^{\text{SEQ}}$ on top of OSTRICH [20] and Princess [38]. To evaluate the effectiveness of $\text{OSTRICH}^{\text{SEQ}}$, we curate two benchmark suites of constraints generated from both real-world and hand-crafted JavaScript programs, on which we compare $\text{OSTRICH}^{\text{SEQ}}$ with cvc5 , Z3 , Z3-noodler [22], OSTRICH and $\text{Princess}^{\text{ARR}}$. The experimental results show that $\text{OSTRICH}^{\text{SEQ}}$ can solve considerably more constraints generated from both real-world and hand-crafted JavaScript programs than the SOTA string solvers (e.g., cvc5 and Z3), while being largely as efficient as most of them.

Organization. Section 2 presents a motivating example. Section 3 gives the preliminaries. Section 4 defines the logic SeqStr of string sequences. Section 5 presents the decision procedure. Section 6 presents the benchmarks and experiments. Section 7 discusses the related work. The paper is concluded in Section 8.

2 Motivating Example

To motivate and illustrate our approach, we consider a snippet of a Javascript program shown in Listing 1.1. The program is extracted, with slight adaptations, from a real-world example.¹ Here, the function `prepareVersionNo` extracts from the input string `version` a version number in the floating point number format. Specifically, it splits `version` according to the non-numeric characters into a sequence/array `numbers`. If `numbers` contains exactly one element, `numbers[0]` is returned. Otherwise, the function returns the concatenation of `numbers[0]`, the dot, and the string obtained by concatenating the other elements of `numbers`. For instance, if `version = 12a56b23`, the output is `12.5623`.

To verify the functional correctness of `prepareVersionNo`, we specify the following precondition and postcondition.

- The precondition $\text{version} \in [0-9]+([0-9a-zA-Z._/-])^*$ requires that the input string starts with a decimal number, followed by a string of digits, letters, dot `.`, underline `_`, blank symbol, slash `/`, or hyphen `-`.
- As postcondition $\text{result} \in [0-9]+(\.[0-9]+)^?$, the output string starts with a decimal number, possibly followed by a dot `.` and a decimal number.

We apply symbolic execution to verify `prepareVersionNo`. We enumerate the execution paths of `prepareVersionNo` and check for each path that, provided the input string satisfies the precondition, after the execution the output string satisfies the postcondition. There are three execution paths: path-1, taking `numbers.length===1`; path-2, taking `numbers.length>1`; and path-3, in which both conditions are false. As an example, we consider path-2, where the symbolic execution reduces to deciding the satisfiability of the SMT formula in Listing 1.2, where `preReg` and `postReg` denote the regular expressions in the precondition and postcondition respectively, i.e., $[0-9]+([0-9a-zA-Z._/-])^*$ and $[0-9]+(\.[0-9]+)^?$, and `splitReg` represents `[a-zA-Z._/-]`.

¹ <https://github.com/hgoebl/mobile-detect.js/blob/master/mobile-detect.js>

```

1  function prepareVersionNo(version) {
2    // precondition: version in [0-9]+([0-9a-zA-Z._ /-])*
3    numbers = version.split(/[a-z._ \/\-]/i);
4    if (numbers.length === 1) { // path-1
5      result = numbers[0];      }
6    if (numbers.length > 1) { // path-2
7      temp = numbers[0] + ".";
8      numbers1 = numbers.slice(1, numbers.length);
9      result = temp + numbers1.join("");      }
10   return Number(result);
11   // postcondition: result in [0-9]+(\.[0-9]+)?
12 }

```

Listing 1.1. JavaScript code snippet: The motivating example

```

1  ; precondition
2  (assert (str.in_re version preReg))
3  (assert (= numbers (str.splitre version splitReg)))
4  (assert (< 1 (seq.len numbers)))
5  (assert (= temp (str.++ (seq.nth numbers 0) ".")))
6  (assert (= numbers1
7    (seq.extract numbers 1 (- (seq.len numbers) 1))))
8  (assert (= result (str.++ temp (seq.join numbers1 ""))))
9  ; postcondition
10 (assert (not (str.in_re result postReg)))

```

Listing 1.2. Constraints for path-2 in `prepareVersionNo`

The SMT formula involves various operations of string sequences and strings, including sequence length `seq.len`, string split `str.splitre`, sequence read `seq.nth`, string concatenation `str.++`, sequence extract `seq.extract`, sequence join `seq.join`, as well as regular constraints `str.in_re`. While state-of-the-art SMT solvers like Z3 and cvc5 can solve constraints in the generic sequence theory, they do not directly support complex operations that feature mutual transformations between strings and string sequences, such as `str.splitre` and `seq.join`. This motivates us to define an SMT theory of string sequences (`SeqStr`, cf. Section 4) and investigate its decision procedure (Section 5).

Due to the general undecidability of `SeqStr`, we focus on the straight-line fragment, into which the constraint in Listing 1.2 falls. We encode string sequences as strings, and, accordingly, operations involving string sequences as string operations. In the example, the resulting constraint contains a complex combinations of `str.replace_re_all`, `str.substr` and `str.len` and cannot be solved by Z3 or cvc5. (Z3 returns `unknown` as it does not support the `str.replace_re_all` function well; cvc5 fails to solve the problem in 300 seconds.) Our new solver `OSTRICHSEQ` solves Listing 1.2 in less than 1 second.

3 Preliminaries

Throughout the paper, let $\mathbb{N}$ denote the set of natural numbers. For $1 \leq n \in \mathbb{N}$, let $[n] := \{1, \dots, n\}$, and for $m < n \in \mathbb{N}$, let $[m, n] := \{j \mid m \leq j \leq n\}$.

We fix an alphabet Σ , i.e., a finite set of letters. A *string* over Σ is a finite sequence of letters from Σ . We use Σ^* to denote the set of strings over Σ and ε to denote the empty string. A *language* over Σ is a subset of Σ^* . We will use $L_1, L_2, \dots$ to denote languages. For two languages L_1, L_2 , $L_1 \cup L_2$ denotes the union of L_1 and L_2 , and $L_1 \cdot L_2$ denotes the concatenation of L_1 and L_2 , that is, $\{u_1 \cdot u_2 \mid u_1 \in L_1, u_2 \in L_2\}$. For a language L and $n \in \mathbb{N}$, we define L^n , the *iteration* of L for n times, inductively as: $L^0 = \{\varepsilon\}$ and $L^n = L \cdot L^{n-1}$ for $n > 0$. We also use L^* to denote the iteration of L for arbitrarily many times, that is, $L^* = \bigcup_{n \in \mathbb{N}} L^n$. Moreover, let $L^+ = \bigcup_{n > 0} L^n$.

Regular expressions over Σ are defined in a standard way, i.e.,

$$e \stackrel{\text{def}}{=} \emptyset \mid \varepsilon \mid a \mid e + e \mid e \cdot e \mid e^*, \text{ where } a \in \Sigma.$$

The language $\mathcal{L}(e)$ of e is defined inductively as: $\mathcal{L}(\emptyset) = \emptyset$, $\mathcal{L}(\varepsilon) = \{\varepsilon\}$, $\mathcal{L}(a) = \{a\}$, $\mathcal{L}(e_1 + e_2) = \mathcal{L}(e_1) \cup \mathcal{L}(e_2)$, $\mathcal{L}(e_1 \cdot e_2) = \mathcal{L}(e_1) \cdot \mathcal{L}(e_2)$, $\mathcal{L}(e^*) = (\mathcal{L}(e_1))^*$.

As $+$ is associative and commutative, we write $(e_1 + e_2) + e_3$ as $e_1 + e_2 + e_3$ for brevity. We use the abbreviation $e^+ \equiv e \cdot e^*$. Moreover, for $\Gamma = \{a_1, \dots, a_n\} \subseteq \Sigma$, we may overload Γ (resp. Γ^*) as the regular expression $a_1 + \dots + a_n$ (resp. $(a_1 + \dots + a_n)^*$). In addition, $|e|$ denotes the number of symbols occurring in e .

Definition 1 (Finite-state automata). A (nondeterministic) finite-state automaton (NFA) over the alphabet Σ is a tuple $\mathcal{A} = (Q, \Sigma, \delta, I, F)$, where Q is a finite set of states, $I \subseteq Q$ is a set of initial states, $F \subseteq Q$ is a set of final states, and $\delta \subseteq Q \times \Sigma \times Q$ is the transition relation.

Given an input string $w \in \Sigma^*$, a *run* of $\mathcal{A}$ on w is a sequence $q_0 a_1 q_1 \dots a_n q_n$ such that $q_0 \in I$, $w = a_1 \dots a_n$ and $(q_{j-1}, a_j, q_j) \in \delta$ for every $j \in [n]$. The run is *accepting* if $q_n \in F$. A string w is *accepted* by $\mathcal{A}$ if there is an accepting run of $\mathcal{A}$ on w . In particular, the empty string ε is accepted by $\mathcal{A}$ if $I \cap F \neq \emptyset$. The set of strings accepted by $\mathcal{A}$, i.e., the language *recognized* by $\mathcal{A}$, is denoted by $\mathcal{L}(\mathcal{A})$. The *size* $|\mathcal{A}|$ of $\mathcal{A}$ is defined as the cardinality of the set Q of states, which will be used when the computational complexity is concerned.

For convenience, $(q, a, q') \in \delta$ may be written as $q \xrightarrow[a]{a} q'$ or $q \xrightarrow{a} q'$, and for each letter $a \in \Sigma$, let $\delta^{(a)}$ denote the relation $\{(q, q') \mid (q, a, q') \in \delta\}$.

Definition 2 (Finite-state transducer). A (nondeterministic) finite transducer (NFT) $\mathfrak{T}$ is an extension of NFA with outputs. Formally, an NFT $\mathfrak{T}$ is a tuple $(Q, \Sigma, \delta, I, F)$, where Q, Σ, I, F are the same as in NFA and the transition relation δ is a finite subset of $Q \times \Sigma \times Q \times \Sigma^*$. Similarly to NFA, for readability, we write a transition $(q, a, q', u) \in \delta$ as $q \xrightarrow[\delta]{a, u} q'$ or $q \xrightarrow{a, u} q'$. The size of an NFT $\mathfrak{T}$, denoted by $|\mathfrak{T}|$, is defined as the sum of the sizes of the transitions of $\mathfrak{T}$, where the size of a transition $q \xrightarrow{a, u} q'$ is defined as $|u| + 3$.

A run of $\mathfrak{T}$ over a string $w = a_1 \dots a_n$ is a sequence of transitions

$$q_0 \xrightarrow{a_1, u_1} q_1 \xrightarrow{a_2, u_2} q_2 \dots q_{n-1} \xrightarrow{a_n, u_n} q_n$$

where $q_0 \in I$. Similary to NFA, the run is *accepting* if $q_n \in F$. The string $u_1 \cdots u_n$ is called the *output* of the run. The transduction $\mathcal{T}(\mathfrak{T}) \subseteq (\Sigma^*, \Sigma^*)$ defined by $\mathfrak{T}$ is the set of string pairs (w, u) such that there is an accepting run of $\mathfrak{T}$ on w , with the output u .

We will also use standard quantifier-free/existential *linear integer arithmetic* (LIA) formulas, which are typically ranged over by ϕ, φ , etc.

4 A Theory of String Sequences (SeqStr)

A *sequence* over the set X is $(a_0, \dots, a_{n-1})$, where $a_i \in X$ for each $i \in [0, n-1]$, and n is the *length* of the sequence. We shall focus on *string sequences*, namely, sequences over Σ^* . We use $()$ to denote the empty sequence.

Operations on string sequences and strings. Let us assume two string sequences $s_1 = (u_0, \dots, u_{m-1})$ and $s_2 = (v_0, \dots, v_{n-1})$.

- $s_1 \cdot s_2$ is the concatenation of s_1 and s_2 , i.e., $(u_0, \dots, u_{m-1}, v_0, \dots, v_{n-1})$.
- $\text{nth}(s_1, i)$ is the i -th element of s_1 if $i \in [0, m-1]$, and undefined otherwise.
- $\text{join}_v(s_1)$ joins the elements in s_1 with v as the intermediate string, i.e., $\text{join}_v(s_1) = u_0 v \cdots v u_{m-1}$.
- $s_1[i \rightarrow u]$ replaces the i -th element of s_1 by u if $i \in [0, m-1]$, and is s_1 otherwise, i.e.

$$s_1[i \rightarrow u] = \begin{cases} (u_0, \dots, u_{i-1}, u, u_{i+1}, \dots, u_{m-1}), & \text{if } i \in [0, m-1]; \\ s_1, & \text{otherwise.} \end{cases}$$

- $\text{filter}_e(s_1)$ filters out all elements in s_1 that do not match the regular expression e , i.e., $\text{filter}_e(s_1) = (u_{i_1}, \dots, u_{i_k})$ such that $0 \leq i_1 < \dots < i_k \leq m-1$ and for all $j \in [0, m-1]$, $u_j \in \mathcal{L}(e)$ iff $j \in \{i_1, \dots, i_k\}$. In particular, if none of $u_0, \dots, u_{m-1}$ are in $\mathcal{L}(e)$, then $\text{filter}_e(s_1) = ()$.
- $\text{split}_e(u)$ splits a string u into a sequence of substrings according to the regular expression e , i.e., $\text{split}_e(u) = (u'_1, \dots, u'_k)$, where $u = u'_1 v_1 u'_2 v_2 \cdots u'_{k-1} v_{k-1} u'_k$ such that (1) for each $i \in [k-1]$, v_i is the leftmost and longest substring of $u'_i v_i u'_{i+1}$ that matches e , and (2) u'_k does not contain any substring that matches e . In particular, if u does not contain any substring that matches e , then $\text{split}_e(u) = (u)$.
- $s_1[i, j]$ is defined as follows: if $i \in [0, m-1]$ and $j \geq 1$, then $s_1[i, j]$ is the subsequence $(u_i, \dots, u_{\min(i+j-1, m-1)})$; if $i \in [0, m-1]$ and $j = 0$, then $s_1[i, j]$ is the empty sequence $()$; if $i \notin [0, m-1]$ or $j < 0$, then $s_1[i, j]$ is undefined.
- $\text{matchAll}_e(u)$ extracts the substrings of u that match the regular expression e , i.e., $\text{matchAll}_e(u) = (v_1, \dots, v_{k-1})$ such that $u = u'_1 v_1 u'_2 v_2 \cdots u'_{k-1} v_{k-1} u'_k$ and (1) for each $i \in [k-1]$, v_i is the leftmost and longest occurrence of e in $u'_i v_i u'_{i+1}$, and (2) u'_k does not contain any substring that matches e .

The logic SeqStr of string sequences. SeqStr has three data types: **Int** (integer), **Str** (strings), and **Seq** (sequences). We use $u, v, \dots$ (resp. $\mathbf{u}, \mathbf{v}, \dots$) to denote string constants (resp. variables); $m, n, \dots$ (resp. $\mathbf{m}, \mathbf{n}, \dots$) to denote integer constants

(resp. variables); $s, t, \dots$ (resp. $\mathfrak{s}, \mathfrak{t}, \dots$) to denote sequence constants (resp. variables); and e to denote regular expressions.

The syntax of SeqStr is defined as follows.

$$\begin{array}{ll}
it \stackrel{\text{def}}{=} m \mid \mathfrak{m} \mid it + it \mid it - it \mid \text{strlen}(strt) \mid \text{seqlen}(seqt) & \text{integer terms} \\
strt \stackrel{\text{def}}{=} u \mid \mathfrak{u} \mid strt \cdot strt \mid \text{nth}(seqt, it) \mid \text{join}_u(seqt) & \text{string terms} \\
seqt \stackrel{\text{def}}{=} (strt) \mid s \mid \mathfrak{s} \mid seqt \cdot seqt \mid seqt[it \rightarrow strt] \mid \text{filter}_e(seqt) \mid \\
\quad seqt[it, it] \mid \text{split}_e(strt) \mid \text{matchAll}_e(strt) & \text{sequence terms} \\
\varphi \stackrel{\text{def}}{=} it \bowtie it \mid seqt = seqt \mid strt = strt \mid strt \in e \mid \varphi \wedge \varphi & \text{formulas}
\end{array}$$

where $\bowtie \in \{=, \neq, <, \leq, >, \geq\}$.

Here, it , $seqt$ and $strt$ denote integer, sequence and string terms, respectively. In particular, the integer terms are linear arithmetic expressions where $\text{seqlen}(\mathfrak{s})$ (resp. $\text{strlen}(\mathfrak{u})$) denotes the length of a sequence $\mathfrak{s}$ (resp. string $\mathfrak{u}$); the sequence terms are constructed from a singleton sequence, sequence constants and sequence variables by applying the aforementioned sequence/string operations. A SeqStr formula is a conjunction of atomic formulas, each of which is of the form $it_1 \bowtie it_2$ where $\bowtie \in \{=, \neq, <, \leq, >, \geq\}$, a sequence equality $seqt_1 = seqt_2$, or a string equality $strt_1 = strt_2$. Note that sequence and string inequalities can be expressed as well. For instance (using disjunctions and quantifiers for sake of presentation):

$$\begin{aligned}
\mathfrak{x} \neq \mathfrak{y} &\equiv \exists \mathfrak{x}_1, \mathfrak{x}_2, \mathfrak{y}_2. \bigvee_{a, b \in \Sigma, a \neq b} (\mathfrak{x} = \mathfrak{x}_1 a \mathfrak{x}_2 \wedge \mathfrak{y} = \mathfrak{x}_1 b \mathfrak{y}_2) \vee \exists \mathfrak{z}. \bigvee_{a \in \Sigma} (\mathfrak{x} = \mathfrak{y} a \mathfrak{z} \vee \mathfrak{y} = \mathfrak{x} a \mathfrak{z}), \\
\alpha \neq \beta &\equiv (\exists \alpha_1, \alpha_2, \beta_2, \mathfrak{z}_1, \mathfrak{z}_2. \alpha = \alpha_1 \cdot (\mathfrak{z}_1) \cdot \alpha_2 \wedge \beta = \alpha_1 \cdot (\mathfrak{z}_2) \cdot \beta_2 \wedge \mathfrak{z}_1 \neq \mathfrak{z}_2) \vee \\
&\quad \exists \gamma, \mathfrak{z}. \alpha = \beta \cdot (\mathfrak{z}) \cdot \gamma \vee \beta = \alpha \cdot (\mathfrak{z}) \cdot \gamma.
\end{aligned}$$

Proposition 1. *The satisfiability of SeqStr is undecidable.*

We define the *straight-line fragment* SeqStr_{SL} of SeqStr. It is easy to see that by introducing fresh variables, we can rewrite a SeqStr formula to a form in which the left-hand side of each equality $seqt_1 = seqt_2$ (resp. $strt_1 = strt_2$) is a sequence (resp. string) variable. Note that the original and rewritten formulas are equisatisfiable. Henceforth, we assume that all sequence/string equalities satisfy this constraint unless otherwise stated.

Definition 3 (Straight-line fragment). *Given a SeqStr formula φ , the dependency graph G_φ of φ is a directed graph (V, E) such that V is the set of string and sequence variables in φ , and $(v_1, v_2) \in E$ iff $v_1 = \mathbf{rhs}$ is a sequence or string equality in φ and v_2 occurs in $\mathbf{rhs}$ except for integer terms.*

φ is *straight-line* if φ is disjunction-free, and each string/sequence variable occurs as the left-hand sides of equalities at most once, moreover, and G_φ is acyclic.

Example 1. $\mathfrak{s} = \text{split}_e(\mathfrak{u}) \wedge \mathfrak{t} = \mathfrak{s}[1, \text{seqlen}(\mathfrak{s}) - 1]$ is straight-line, while $\mathfrak{s} = \mathfrak{s}[1, \text{seqlen}(\mathfrak{s}) - 1]$ is not, since there is a self-dependency on $\mathfrak{s}$.

5 The Decision Procedure

Theorem 1. *The satisfiability of $\text{SeqStr}_{\text{SL}}$ is decidable.*

The general idea of the decision procedure is to encode each string sequence as a string, based on which all operations in $\text{SeqStr}_{\text{SL}}$ can be transformed into string operations. The decision procedure utilizes the framework [19] for solving straight-line string constraints with integer data type. There are, however, certain technical challenges. For instance, $\text{seqt}[it \rightarrow str]$ and $\text{seqt}[it, it]$ cannot be captured directly by standard string operations so new ones have to be provided and their pre-images need to be computed.

5.1 From string sequences to strings

We fix a fresh symbol $\dagger \notin \Sigma$ and let $\Sigma' := \Sigma \cup \{\dagger\}$. Each string sequence $s = (u_1, \dots, u_m)$ over Σ is encoded as a string $\text{enc}(s) := \dagger u_1 \dagger \dots \dagger u_m \dagger$ over Σ' . Note that the strings resulted from the encoding should match the regular expression $\dagger(\Sigma^* \dagger)^*$ and the string $\dagger$ encodes the empty sequence.

With this encoding of sequences as strings, each string sequence operation is also transformed into the corresponding *string operation*.

sequence concatenation $s_1 \cdot s_2$: string concatenation $\text{enc}(s_1) \cdot \text{enc}(s_2)$.

sequence write $s[i \rightarrow u]$: $\text{write}(\text{enc}(s), i + 1, u)$, which replaces the substring of $\text{enc}(s)$ between the $(i + 1)$ -th occurrence of $\dagger$ and the $(i + 2)$ -th occurrence of $\dagger$, by u , if $i \in [0, n - 2]$, where n is the number of occurrences of $\dagger$ in $\text{enc}(s)$, and is undefined otherwise (i.e. $i \notin [0, n - 2]$). Note that this operation is not a typical string replace operation. (Recall that the indices of elements in sequences start from 0 and n , i.e. the number of occurrences of $\dagger$ in $\text{enc}(s)$, is one more than the length of s .)

sequence filter operation $\text{filter}_e(s)$: $\text{filter}_e(\text{enc}(s))$, which removes every substring of $\text{enc}(s)$ between two consecutive occurrences of $\dagger$ that does not match the regular expression e . (For instance, if $e = a^*b$ and $s = (ab, ac, aab)$, then $\text{filter}_e(\text{enc}(s)) = \dagger ab \dagger aab \dagger$.)

subsequence operation $s[i, j]$: $\text{subseq}(\text{enc}(s), i + 1, j)$, which keeps only the substring of $\text{enc}(s)$ between the $(i + 1)$ -th occurrence and the $\min(i + j, n - 1)$ -th occurrence of $\dagger$, if $i \in [0, n - 2]$, where n is the number of occurrences of $\dagger$ in $\text{enc}(s)$, and is undefined otherwise (i.e. $i \notin [0, n - 2]$).

split_e(u): string operation $\text{splitstr}_e(u)$ that transforms $u = u'_1 v_1 u'_2 \dots u'_{k-1} v_{k-1} u'_k$ into $\dagger u'_1 \dagger u'_2 \dots u'_{k-1} \dagger u'_k \dagger$, where for each $i \in [k - 1]$, v_i is the leftmost and longest matching of e in $u'_i v_i u'_{i+1}$, moreover, u'_k does not contain any substring that matches e .

matchAll_e(u): $\text{matchAllstr}_e(u)$ that transforms $u = u'_1 v_1 u'_2 v_2 \dots u'_{k-1} v_{k-1} u'_k$ into $\dagger v_1 \dagger \dots \dagger v_{k-1} \dagger$, where for each $i \in [k - 1]$, v_i is the leftmost and longest occurrence of e in $u'_i v_i u'_{i+1}$, moreover, u'_k does not contain any substring that matches e .

sequence read operation $\text{nth}(s, i + 1)$: $\text{elem}(\text{enc}(s), i + 1)$ that keeps only the substring between the $(i + 1)$ -th occurrence and the $(i + 2)$ -th occurrence of

$\dagger$, if $i \in [0, n - 2]$, where n is the number of occurrences of $\dagger$ in $\text{enc}(s)$, and is undefined otherwise (i.e. $i \notin [0, n - 2]$). (For instance, if $s = (ab, ac)$ and $i = 0$, then $\text{elem}(\text{enc}(s), 1) = \text{elem}(\dagger ab \dagger ac \dagger, 1) = ab$.)

sequence join operation $\text{join}_u(s)$: $\text{join}_u(\text{enc}(s))$ that removes the first and the last occurrences of $\dagger$ and replaces all the remaining occurrences of $\dagger$ by u .

sequence length operation $\text{seqlen}(s)$: $\text{seqlen}(\text{enc}(s)) - 1$ where $\text{seqlen}(\text{enc}(s))$ counts the number of occurrences of $\dagger$ in $\text{enc}(s)$.

Hence we obtain atomic string constraints of the following forms,

$$\begin{aligned} & \mathbf{x} = u \mid \mathbf{x} = \eta \mid \mathbf{x} = \eta \cdot \mathbf{z} \mid \mathbf{x} = \mathfrak{T}(\eta) \mid \mathbf{z} = \text{write}(\mathbf{x}, it, \eta) \mid \eta = \text{filter}_e(\mathbf{x}) \mid \\ & \eta = \text{splitstr}_e(\mathbf{x}) \mid \eta = \text{subseq}(\mathbf{x}, it_1, it_2) \mid \eta = \text{matchAllstr}_e(\mathbf{x}) \mid \eta = \text{elem}(\mathbf{x}, it) \mid \\ & \eta = \text{join}_u(\mathbf{x}) \mid it = \text{strlen}(\mathbf{x}) \mid it = \text{seqlen}(\mathbf{x}), \end{aligned}$$

where $\mathfrak{T}$ is a finite-state transducer.

We denote by $\mathbf{XStr}$ the class of string constraints which are positive Boolean combinations (no negation) of the atomic string constraints, and $\mathbf{XStr}_{\text{SL}}$ denotes its straight-line fragment. For each formula φ in $\text{SeqStr}_{\text{SL}}$, $\text{enc}(\varphi)$ is the resulting formula in $\mathbf{XStr}$. We have the following result.

Proposition 2. *For each constraint φ in SeqStr , φ and $\text{enc}(\varphi)$ are equisatisfiable. Moreover, if φ is in $\text{SeqStr}_{\text{SL}}$, then $\text{enc}(\varphi)$ is in $\mathbf{XStr}_{\text{SL}}$.*

Example 2. Consider the $\text{SeqStr}_{\text{SL}}$ constraint $\varphi := (\mathfrak{s}_1 = \mathfrak{s}_0 \cdot (u, v) \wedge \text{seqlen}(\mathfrak{s}_1) < 2)$ where $\mathfrak{s}_0$ is a string variable and u, v are string constants. Then, its string encoding is $\text{enc}(\varphi) := (\text{enc}(\mathfrak{s}_1) = \text{enc}(\mathfrak{s}_0) \cdot \dagger u \dagger v \dagger \wedge \text{seqlen}(\text{enc}(\mathfrak{s}_1)) - 1 < 2)$, which is unsatisfiable, since there are at least three occurrences of $\dagger$ in $\text{enc}(\mathfrak{s}_1)$. $\square$

The satisfiability of $\text{SeqStr}_{\text{SL}}$ is now reduced to that of $\mathbf{XStr}_{\text{SL}}$. It is easy to observe that the operations filter_e , splitstr_e , matchAllstr_e and join_u can all be represented by a finite-state transducer, so the results in [19] are directly applicable. In the next section, we show that the pre-images of the remaining string operations, i.e., write , subseq , elem and seqlen , can be effectively computed.

5.2 Computing the pre-images of write , subseq , elem and seqlen

[19] introduced cost-enriched finite automata (CEFA for short) and showed that the pre-images of all the considered string operations involving the string and integer data types only can be effectively computed. The OSTRICH framework essentially informs that once the pre-image is computable, the straight-line fragment can be solved by propagating the constraints encoded in CEFA backwards. Our decision procedure for $\mathbf{XStr}_{\text{SL}}$ constraints follow this approach.

We first recall the definition of CEFAs. Intuitively, CEFAs add write-only cost registers to finite state automata, where “write-only” means that the cost registers can only be written/updated but cannot be read, i.e., they cannot be used in the guards of the transitions.

A CEFA $\mathcal{A}$ over Σ' is a tuple $(R, Q, \Sigma', \delta, I, F)$ where

- $R = \{r_1, \dots, r_k\}$ is a finite set of registers,

- Q, I, F are the same as in NFA, i.e., the set of states, the set of initial states, and the set of final states, respectively,
- $\delta \subseteq Q \times \Sigma' \times Q \times \mathbb{Z}^R$ is a transition relation, where $\mathbb{Z}^R$ denotes the updates on the values of registers. (Recall that $\Sigma' := \Sigma \cup \{\dagger\}$ is the extended alphabet.)

We write $R_{\mathcal{A}}$ for the set of registers of $\mathcal{A}$. Typically, $R_{\mathcal{A}}$ is represented as a vector $(r_1, \dots, r_k)$. Accordingly, we write assignments $r_i := r_i + v_i$ for all $i \in [k]$ simply as $\vec{v} = (v_1, \dots, v_k)$, i.e., r_i is to be incremented by v_i for each $i \in [k]$. We also write a transition $(q, a, q', \vec{v}) \in \delta$ as $q \xrightarrow[\vec{v}]{a} q'$. In particular, (q, a, q') stands for the transition $(q, a, q', ())$ without any updates.

Let $\mathcal{A} = (R, Q, \Sigma', \delta, I, F)$ be a CEFA. A *run* of $\mathcal{A}$ on a string $w = a_1 \dots a_n$ over Σ' is a sequence $q_0 \xrightarrow[\vec{v}_1]{a_1} q_1 \dots q_{n-1} \xrightarrow[\vec{v}_n]{a_n} q_n$ such that $q_0 \in I$ and $q_{i-1} \xrightarrow[\vec{v}_i]{a_i} q_i$ for each $i \in [n]$. It is *accepting* if $q_n \in F$, and the vector $\vec{c} = \sum_{j \in [n]} \vec{v}_j$ is defined as the *cost* of an accepting run. (Note that all registers are initialized to zero.) We write $\vec{c} \in \mathcal{A}(w)$ if there is an accepting run of $\mathcal{A}$ on w whose cost is $\vec{c}$.

The semantics of a CEFA $\mathcal{A}$, denoted by $\mathcal{L}(\mathcal{A})$, is defined as the set of pairs $\{(w, \vec{c}) \mid \vec{c} \in \mathcal{A}(w)\}$. In particular, if $I \cap F \neq \emptyset$, then $(\varepsilon, \vec{0}) \in \mathcal{L}(\mathcal{A})$. Moreover, the *output* of a CEFA $\mathcal{A}$, denoted by $\mathcal{O}(\mathcal{A})$, is defined as $\{\vec{c} \mid \exists w. \vec{c} \in \mathcal{A}(w)\}$.

The following example illustrates the backward propagation.

Example 3. Consider the equality $\mathbf{v} = f(\mathbf{u}_1, it_1, \mathbf{u}_2, it_2)$ (where $\mathbf{u}_1, \mathbf{u}_2$ are string variables, it_1, it_2 are integer terms, and f is a function) and the CEFA $\mathcal{A} = (R, Q, \Sigma', \delta, I, F)$, where $R = \{r_1\}$. Then an R -cost-enriched pre-image of $\mathcal{A}$ under f , denoted by $f_R^{-1}(\mathcal{A})$, is a pair $(\mathcal{R}, t)$ where $\mathcal{R} \subseteq (\Sigma')^* \times (\mathbb{Z} \times \mathbb{Z}) \times (\Sigma')^* \times (\mathbb{Z} \times \mathbb{Z})$, t is an LIA term over $\{r_1^{(1)}, r_1^{(2)}\}$, and $\mathcal{R}$ satisfies that for every $(v, n') \in \mathcal{L}(\mathcal{A})$, there is $(u_1, n_1, n'_1, u_2, n_2, n'_2) \in \mathcal{R}$ such that $f(u_1, n_1, u_2, n_2) = v$, and $n' = t[n'_1/r_1^{(1)}, n'_2/r_1^{(2)}]$. We say that a pre-image $f_R^{-1}(\mathcal{A}) = (\mathcal{R}, t)$ is CEFA-definable if $\mathcal{R}$ can be expressed as a finite collection of CEFA tuples $(\mathcal{B}_{j,1}, \mathcal{B}_{j,2})$ with $j \in [k]$ such that $R_{\mathcal{B}_{j,1}} = (r'_{1,1}, r_1^{(1)})$, $R_{\mathcal{B}_{j,2}} = (r'_{2,1}, r_1^{(2)})$, and $\mathcal{R} = \bigcup_{j \in [k]} \mathcal{L}(\mathcal{B}_{j,1}) \times \mathcal{L}(\mathcal{B}_{j,2})$. Intuitively, $r'_{1,1}$ and $r'_{2,1}$ are the fresh registers corresponding to the two integer parameters, and $r_1^{(1)}, r_1^{(2)}$ are the fresh registers introduced in $\mathcal{B}_{j,1}$ and $\mathcal{B}_{j,2}$ respectively for computing r_1 according to t . Then the backward propagation of the constraint $\mathbf{v} \in \mathcal{A}$ under f nondeterministically chooses a tuple $(\mathcal{B}_{j,1}, \mathcal{B}_{j,2})$, removes $\mathbf{v} = f(\mathbf{u}_1, it_1, \mathbf{u}_2, it_2)$ from the string constraint, and adds $\mathbf{u}_1 \in \mathcal{B}_{j,1} \wedge \mathbf{u}_2 \in \mathcal{B}_{j,2} \wedge r_1 = t \wedge it_1 = r'_{1,1} \wedge it_2 = r'_{2,1}$ to the string constraint.

We now show how to compute the pre-images of **write**, **subseq**, **elem** and **seqen**. Note that the pre-image computation utilizes an NFA $\mathcal{A}_0$ (resp. $\mathcal{A}_1$) that formats the strings that represent the string sequences (resp. strings), namely, $\mathcal{A}_0$ (resp. $\mathcal{A}_1$) recognizes the language $\dagger(\Sigma\dagger)^*$ (resp. Σ^*).

Pre-image of write. Let $\mathcal{A} = (R, Q, \Sigma', \delta, I, F)$ be a CEFA. Intuitively, $\text{write}^{-1}(\mathcal{A})$ is computed as the collection $((\mathcal{B}_{(p,q)} \cap \mathcal{A}_0, \mathcal{A}_{R_2/R}[p, q] \cap \mathcal{A}_1), t_{(p,q)})_{(p,q) \in Q \times Q}$, where $\mathcal{B}_{(p,q)}$, $\mathcal{A}_{R_2/R}[p, q]$ and $t_{(p,q)}$ are constructed as follows.

- $\mathcal{B}_{(p,q)}$ is constructed by the following procedure.
 - We create two copies of R , say R_1, R_2 . Moreover, we introduce a fresh cost register, say r' , to count the number of occurrences of $\dagger$.
 - We run $\mathcal{A}$ on the input string and increase the cost register r' each time when $\dagger$ is read. Moreover, the registers in R_1 , instead of R , are updated in the transitions.
 - When the current symbol is $\dagger$, we nondeterministically choose to stop increasing r' and pause the running of $\mathcal{A}$ at the state p . Let us call this position as the pause position.
 - Finally, when reading the first $\dagger$ after the pause position, we resume the run of $\mathcal{A}$ at q , where the registers in R_1 (but not r') are updated.
- $\mathcal{A}_{R_2/R}[p, q]$ is obtained from $\mathcal{A}[p, q]$ by replacing each register in R with the corresponding copy in R_2 , where $\mathcal{A}[p, q]$ is the sub-automaton of $\mathcal{A}$ that accepts the strings starting from the state p and ending at the state q .
- $t_{(p,q)} := (t_{(p,q),r})_{r \in R}$ and $t_{(p,q),r} = r^{(1)} + r^{(2)}$ for each $r \in R$.

Note that in the backward propagation of $\mathcal{A}$ with respect to an equality $\mathfrak{z} = \text{write}(\mathfrak{x}, it, \mathfrak{y})$, the constraint $r' = it$ is added to the LIA constraint to assert that r' is equal to the index it . Formally, $\text{write}^{-1}(\mathcal{A})$ is a finite collection of $(\mathcal{B}_{(p,q)} \cap \mathcal{A}_0, \mathcal{A}_{R_2/R}[p, q] \cap \mathcal{A}_1, t_{(p,q)})$, where $(p, q) \in Q \times Q$, and $\mathcal{B}_{(p,q)} = (R_1 \cup \{r'\}, Q', \Sigma', \delta', I', F')$ such that $Q' = Q \times \{pre, idle, post\}$, $I' = I \times \{pre\}$, $F' = F \times \{post\}$, and δ' comprises,

- $((q_1, pre), a, (q_2, pre), (\vec{v}, 0))$ such that $(q_1, a, q_2, \vec{v}) \in \delta$ and $a \in \Sigma$ (thus $a \neq \dagger$),
- $((q_1, pre), \dagger, (q_2, pre), (\vec{v}, 1))$ such that $(q_1, \dagger, q_2, \vec{v}) \in \delta$,
- $((q_1, pre), \dagger, (p, idle), (\vec{v}, 1))$ such that $(q_1, \dagger, p, \vec{v}) \in \delta$,
- $((p, idle), a, (p, idle), (\vec{0}, 0))$ such that $a \in \Sigma$,
- $((p, idle), \dagger, (q_2, post), (\vec{v}, 0))$ such that $(q, \dagger, q_2, \vec{v}) \in \delta$,
- $((q_1, post), a, (q_2, post), (\vec{v}, 0))$ such that $(q_1, a, q_2, \vec{v}) \in \delta$ and $a \in \Sigma$,
- $((q_1, post), \dagger, (q_2, post), (\vec{v}, 0))$ such that $(q_1, \dagger, q_2, \vec{v}) \in \delta$.

From the construction of $\mathcal{B}_{(p,q)}$, we can observe that the pair (p, q) should satisfy that in $\mathcal{A}$, there is a $\dagger$ -transition into p and a $\dagger$ -transition out of q . Moreover, q should be reachable from p according to the construction of $\mathcal{A}_{R_2/R}[p, q] \cap \mathcal{A}_1$.

Example 4. Let us consider the constraint $\mathfrak{v} = \text{write}(\mathfrak{u}, 1, \mathfrak{y}) \wedge \mathfrak{v} \in \dagger(a^+\dagger)^* \wedge \text{strlen}(\mathfrak{v}) \geq 4$. Let $\mathcal{A} = (\{r_1\}, \{q_0, q_1, q_2\}, \Sigma', \delta, \{q_0\}, \{q_1\})$ be the CEFA, where the register r_1 is used to record the string length and δ comprises the transitions $q_0 \xrightarrow[(1)]{\dagger} q_1 \xrightarrow[(1)]{a} q_2 \xrightarrow[(1)]{a} q_2 \xrightarrow[(1)]{\dagger} q_1$. Let us consider the pairs (p, q) in $\mathcal{A}$ satisfying that there is a $\dagger$ -transition into p and a $\dagger$ -transition out of q , moreover, q should be reachable from p . It is not hard to observe that there is exactly one such pair, that is, (q_1, q_2) . Therefore, $\text{write}^{-1}(\mathcal{A})$ comprises exactly one tuple $(\mathcal{B}_{(q_1, q_2)} \cap \mathcal{A}_0, \mathcal{A}_{R_2/R}[q_1, q_2] \cap \mathcal{A}_1, (r_1^{(1)} + r_1^{(2)}))$ such that $\mathcal{B}_{(q_1, q_2)} = (\{r_1^{(1)}, r'\}, Q', \Sigma', \delta', I', F')$ where

- $Q' = \{q_0, q_1, q_2\} \times \{pre, idle, post\}$, $I' = \{(q_0, pre)\}$, $F' = \{(q_1, post)\}$, and

- δ' comprises the following transitions
 - $((q_1, pre), a, (q_2, pre), (1, 0)), ((q_2, pre), a, (q_2, pre), (1, 0)),$
 - $((q_0, pre), \dagger, (q_1, pre), (1, 1)), ((q_2, pre), \dagger, (q_1, pre), (1, 1)),$
 - $((q_0, pre), \dagger, (q_1, idle), (1, 1))$ (since $(q_0, \dagger, q_1, 1) \in \delta$),
 - $((q_1, idle), a', (q_1, idle), (0, 0))$ such that $a' \in \Sigma$,
 - $((q_1, idle), \dagger, (q_1, post), (1, 0))$ (since $(q_2, \dagger, q_1, 1) \in \delta$),
 - $((q_1, post), a, (q_2, post), (1, 0))$ and $((q_2, post), a, (q_2, post), (1, 0)),$
 - $((q_2, post), \dagger, (q_1, post), (1, 0)).$

To illustrate how $\mathcal{B}_{(q_1, q_2)}$ and $\mathcal{A}_{R_2/R}[q_1, q_2]$ work, we can see that $(\dagger aa \dagger a \dagger, 6)$ is accepted by $\mathcal{A}$, moreover, for any $b \in \Sigma$,

- $(\dagger ab \dagger a \dagger, 4, 1)$ is accepted by $\mathcal{B}_{(q_1, q_2)}$, witnessed by the following sequence of transitions,

$$\begin{aligned} (q_0, pre) &\xrightarrow[(1,1)]{\dagger} (q_1, idle) \xrightarrow[(0,0)]{a} (q_1, idle) \xrightarrow[(0,0)]{b} (q_1, idle) \\ &\xrightarrow[(1,0)]{\dagger} (q_1, post) \xrightarrow[(1,0)]{a} (q_2, post) \xrightarrow[(1,0)]{\dagger} (q_1, post), \end{aligned}$$

- and $(aa, 2)$ is accepted by $\mathcal{A}_{R_2/R}[q_1, q_2]$, witnessed by the following sequence of transitions, $q_1 \xrightarrow[(1)]{a} q_2 \xrightarrow[(1)]{a} q_2$.

Thus, $\mathbf{u}$ and $\mathbf{\eta}$ in $\mathbf{v} = \text{write}(\mathbf{u}, 1, \mathbf{\eta})$ can be $\dagger ab \dagger a \dagger$ and aa , respectively. $\square$

Pre-image of subseq. Let $\mathcal{A} = (R, Q, \Sigma', \delta, I, F)$ be a CEFA. Then $\text{subseq}^{-1}(\mathcal{A})$ is computed as $(\mathcal{B} \cap \mathcal{A}_0, t)$ such that $t := \text{true}$ and $\mathcal{B}$ introduces two fresh registers r'_1, r'_2 to count the two numbers of occurrences of $\dagger$ that correspond to the starting position and the length of the subsequence respectively. Moreover, $\mathcal{B}$ simulates the run of $\mathcal{A}$ on the substring representing the subsequence returned by subseq . Formally, $\text{subseq}^{-1}(\mathcal{A})$ is computed as $(\mathcal{B} \cap \mathcal{A}_0, \text{true})$, where $\mathcal{B} = (R \cup \{r'_1, r'_2\}, Q \cup \{pre, post\}, \Sigma', \delta', \{pre\}, \{post\})$ such that $pre, post$ are two fresh states denoting the fact that $\mathcal{B}$ is reading a symbol before and after the subsequence respectively and δ' comprises the following tuples,

- $(pre, a, pre, (\vec{0}, 0, 0))$ such that $a \in \Sigma$ (thus $a \neq \dagger$),
- $(pre, \dagger, pre, (\vec{0}, 1, 0)),$
- $(pre, \dagger, q, (\vec{v}, 1, 0))$ such that $(p, \dagger, q, \vec{v}) \in \delta$ for some $p \in I$,
- $(q, a, q', (\vec{v}, 0, 0))$ such that $(q, a, q', \vec{v}) \in \delta$ and $a \in \Sigma$,
- $(q, \dagger, q', (\vec{v}, 0, 1))$ such that $(q, \dagger, q', \vec{v}) \in \delta$,
- $(q, \dagger, post, (\vec{v}, 0, 1))$ such that $(q, \dagger, q', \vec{v}) \in \delta$ for some $q' \in F$,
- $(post, a, post, (\vec{0}, 0, 0))$ and $(post, \dagger, post, (\vec{0}, 0, 0))$ where $a \in \Sigma$.

Note that in the backward propagation of $\mathcal{A}$ with respect to an equality $\mathbf{z} = \text{subseq}(\mathbf{x}, it_1, it_2)$, the constraint $r'_1 = it_1 \wedge r'_2 = it_2$ is added to the LIA constraint to assert that the value of r'_1 and r'_2 are equal to the beginning index and the length of the subsequence respectively.

Example 5. Let $\mathbf{v} = \text{subseq}(\mathbf{u}, 2, 1) \wedge \mathbf{v} \in \dagger(a^+\dagger)^* \wedge \text{strlen}(\mathbf{v}) \geq 4$ and $\mathcal{A} = (\{r_1\}, \{q_0, q_1, q_2\}, \Sigma', \delta, \{q_0\}, \{q_1\})$ be the CEFA, where the register r_1 is used to record the string length and δ comprises the transitions $q_0 \xrightarrow[(1)]{\dagger} q_1 \xrightarrow[(1)]{a} q_2 \xrightarrow[(1)]{a} q_2 \xrightarrow[(1)]{\dagger} q_1$. Then $\text{subseq}^{-1}(\mathcal{A})$ is $(\mathcal{B} \cap \mathcal{A}_0, \text{true})$ such that

$$\mathcal{B} = (\{r_1^{(1)}, r'_1, r'_2\}, \{q_0, q_1, q_2, \text{pre}, \text{post}\}, \Sigma', \delta', \{\text{pre}\}, \{\text{post}\})$$

where δ' comprises the following transitions

- $(\text{pre}, a', \text{pre}, (0, 0, 0))$ such that $a' \in \Sigma$,
- $(\text{pre}, \dagger, \text{pre}, (0, 1, 0))$,
- $(\text{pre}, \dagger, q_1, (1, 1, 0))$ (since $(q_0, \dagger, q_1, 1) \in \delta$),
- $(q_1, a, q_2, (1, 0, 0))$, $(q_2, a, q_2, (1, 0, 0))$,
- $(q_2, \dagger, q_1, (1, 0, 1))$,
- $(q_2, \dagger, \text{post}, (1, 0, 1))$ (since $(q_2, \dagger, q_1, 1) \in \delta$ and q_1 is an accepting state in $\mathcal{A}$),
- $(\text{post}, a', \text{post}, (0, 0, 0))$ and $(\text{post}, \dagger, \text{post}, (0, 0, 0))$ where $a' \in \Sigma$.

To illustrate how $\mathcal{B}$ works, we can see that $(\dagger a \dagger a a \dagger, 4, 2, 1)$ is accepted by $\mathcal{B}$, witnessed by the following sequence of transitions,

$$\text{pre} \xrightarrow[(0,1,0)]{\dagger} \text{pre} \xrightarrow[(0,0,0)]{a} \text{pre} \xrightarrow[(1,1,0)]{\dagger} q_1 \xrightarrow[(1,0,0)]{a} q_2 \xrightarrow[(1,0,0)]{a} q_2 \xrightarrow[(1,0,1)]{\dagger} \text{post}. \quad \square$$

Pre-image of elem. Let $\mathcal{A} = (R, Q, \Sigma, \delta, I, F)$ be a CEFA. The computation of $\text{elem}^{-1}(\mathcal{A})$ is similar to that of $\text{subseq}^{-1}(\mathcal{A})$. Nevertheless, the construction is slightly different since the output of elem is a string, instead of a sequence. Intuitively, $\text{elem}^{-1}(\mathcal{A})$ is computed as $(\mathcal{B} \cap \mathcal{A}_0, t)$ where $t := \text{true}$, a fresh register r' is introduced to count the number of occurrences of $\dagger$ that corresponds to the position where the element is extracted. Formally, $\text{elem}^{-1}(\mathcal{A})$ is computed as $(\mathcal{B} \cap \mathcal{A}_0, \text{true})$ such that $\mathcal{B} = (R \cup \{r'\}, Q \cup \{\text{pre}, \text{post}\}, \Sigma', \delta', \{\text{pre}\}, \{\text{post}\})$, where δ' comprises the following tuples,

- $(\text{pre}, a, \text{pre}, (\vec{0}, 0))$ such that $a \in \Sigma$ (thus $a \neq \dagger$),
- $(\text{pre}, \dagger, \text{pre}, (\vec{0}, 1))$,
- $(\text{pre}, \dagger, p, (\vec{v}, 1))$ such that $p \in I$,
- $(q, a, q', (\vec{v}, 0))$ such that $(q, a, q', \vec{v}) \in \delta$ and $a \in \Sigma$ (thus $a \neq \dagger$),
- $(q, \dagger, \text{post}, (\vec{0}, 0))$ such that $q \in F$,
- $(\text{post}, a, \text{post}, (\vec{0}, 0))$ and $(\text{post}, \dagger, \text{post}, (\vec{0}, 0))$ where $a \in \Sigma$.

Note that in the backward propagation of $\mathcal{A}$ with respect to an equality $\mathfrak{z} = \text{elem}(\mathbf{r}, it)$, the constraint $r' = it$ is added to the LIA constraint to assert that the value of r' is equal to the index it .

Finally, we remark that the operation $\text{seqlen}(\mathbf{r})$ can be modeled as a CEFA, similarly to the CEFA for strlen in [19].

6 Implementation and Experiments

We implement a solver $\text{OSTRICH}^{\text{SEQ}}$ on top of the string solver OSTRICH [20] and the SMT solver Princess [38]. The implementation consists of approximately 5,000 lines of Scala code. The core functionality, preimage computation for sequence operations such as `split`, `join`, `filter` and `matchAll` are implemented in about 2,000 lines of code and is integrated directly into the OSTRICH framework. To support the representation of sequences, we built a lightweight library of automata which treat the separator as a special transition. The library is implemented in about 1,000 lines of code. The remaining code is used for the integration with OSTRICH and for the implementation of the decision procedures for the theory of sequences.

6.1 Benchmarks

To evaluate the effectiveness of $\text{OSTRICH}^{\text{SEQ}}$, we create two benchmark suites, SEQBASE and SEQEXT.

SEQBASE. The SEQBASE benchmark suite comprises `SeqStr` constraints that contain only the generic sequence operations, that is, sequence operations that can be applied to the elements of any type (not necessarily string), such as sequence length `seqlen`, sequence concatenation, subsequence `seqt[it1, it2]`, sequence read `nth(seqt, it)` and sequence write `seqt[it → strt]`. SEQBASE is designed to facilitate a fairer comparison with the existing solvers since they do not support the string-specific sequence operations. SEQBASE contains 100 `SeqStr` constraints that are generated from three templates. Specifically,

- The first template $u_1 \in e_1 \wedge s = s[0 \rightarrow u_1, 1 \rightarrow u_1] \wedge u_2 = \text{nth}(s, 0) \cdot \text{nth}(s, 1) \wedge u_2 \in e_2 \wedge \text{seqlen}(s_2) < \text{strlen}(u_2)$ is utilized to generate 20 instances, where u_1 is a string variable, s is a sequence variable, s is a sequence constant, and e_1 and e_2 are randomly generated regular expressions. Generally speaking, the template writes the same string to different positions of a sequence and then checks the length property and regular properties of elements.
- The second template $u_1 \in e_1 \wedge s_1 = s[0 \rightarrow u_1, 1 \rightarrow u_2] \wedge s_2 = s_1[0, 2] \cdot s_1[1, 2] \wedge u_3 = \text{nth}(s_2, 0) \cdot \text{nth}(s_2, 1) \cdot \text{nth}(s_2, 2) \cdot \text{nth}(s_2, 3) \wedge u_3 \in e_2 \wedge \text{seqlen}(s_2) < \text{strlen}(u_3) + 1$ is utilized to generate 20 instances, where u_1 and u_2 are string variables, s_1 and s_2 are sequence variables, s is a sequence constant, and e_1 and e_2 are randomly generated regular expressions. Briefly, the template generates a new sequence from an initial sequence by concatenating two overlapping subsequences and then checks the length and regular properties.
- The third template $s_1 = s[m_1 \rightarrow u_1] \wedge s_2 = s_1[m_2 \rightarrow u_2] \wedge \dots \wedge u_1 = \text{nth}(s_1, n_1) \wedge u_2 = \text{nth}(s_2, n_2) \wedge \dots \wedge u_1 \in e_1 \wedge u_2 \in e_2 \wedge \dots$ is utilized to generate 60 instances, where s_1 and s_2 are sequence variables, s is a sequence constant, u_1 and u_2 are string variables, m_1 , m_2 , n_1 , and n_2 are randomly generated concrete integers, and e_1 and e_2 are randomly generated regular expressions. In this template, write ($s[\cdot \rightarrow \cdot]$) and read(nth) operations are performed randomly on a string sequence, and the strings read from the sequence are checked against some regular expressions.

SEQEXT. The SEQEXT benchmark suite comprises SeqStr constraints that use operations that are specialized to string sequences, that is, `filtere`, `matchAlle`, `splite` and `joinu`. There are 60 constraints in SEQEXT, where 19 of them are manually generated for the unit tests of OSTRICH^{SEQ}, and 41 of them are generated from the real-world JavaScript programs collected from GitHub. Moreover, to compare with SMT solvers that do not support the operations specialized to string sequences, we rewrite the constraints in SEQEXT to the basic operations of string theory whenever possible. (`matchAlle` is not rewritten since it can not be simulated using existing string operations.) We partition the constraints in SEQEXT into two categories: No.Mat and Mat. The former contains constraints that do not use `matchAlle`, while the latter may use it.

6.2 Experimental results

We conduct experiments by comparing OSTRICH^{SEQ} with several existing solvers, including `cvc5`, `Z3`, `Z3-noodler`, `OSTRICH` and `PrincessARR` (which refers to an array-theory-based solver implemented within `Princess`). `SeCo` [32] is not included in the comparison due to soundness issues. Additionally, `foldleft` and `map` operations are not used to simulate `join`, as most constraints return unknown under such usage in `Z3`.

On the SEQBASE benchmark, we evaluate OSTRICH^{SEQ} along with sequence solvers `cvc5`, `Z3` and `PrincessARR`. For the SEQEXT benchmark, we rewrite the constraints using only the basic string operations and compare OSTRICH^{SEQ} with string solvers `cvc5`, `Z3`, `Z3-noodler` and `OSTRICH`.

All experiments are run on a 24×3.1 GHz-core server with 185 GB RAM, where each solver is started as a single thread, with a 60s time limit and without the memory limit.

Table 1. Experimental results on SEQBASE.

	<code>cvc5</code>	<code>Z3</code>	<code>Princess^{ARR}</code>	<code>OSTRICH^{SEQ}</code>
sat	33	14	33	33
unsat	50	36	67	67
solved	83	50	100	100
unknown/timeout	17	50	0	0
avg. time (s)	10.77	15.02	4.04	6.24

The results on the SEQBASE can be found in Table 1. We can see that OSTRICH^{SEQ} outperforms `cvc5` and `Z3` on SEQBASE in both the number of solved instances and the solving time per instance. Specifically, OSTRICH^{SEQ} solves all the 100 instances in SEQBASE, while `cvc5` and `Z3` solve 83 and 50 instances respectively. Moreover, OSTRICH^{SEQ} is more efficient than `cvc5` and `Z3`, that is, OSTRICH^{SEQ} spends 6.24 seconds in solving each instance from SEQBASE on average, while `cvc5` and `Z3` spend 10.77 and 15.02 seconds respectively. However, OSTRICH^{SEQ} is slightly slower than `PrincessARR`, which solves all the instances in SEQBASE in 4.04 seconds on average. This is because `PrincessARR` utilizes the optimization techniques that are specialized to the array theory,

making it more efficient for solving constraints that involve only read and write operations.

The results on SEQEXT can be found in Table 2. Since the SOTA sequence solvers do not support the operations filter_e , matchAll_e or split_e directly, the SeqStr constraints is rewritten to string constraints (instead of solving the SeqStr constraints directly) when we compare with them on SEQEXT. Moreover, while we can use $\text{str.replace_re_all}$ to encode filter_e and split_e , the encoding of matchAll_e requires finite-state transducers which have not been supported by the SOTA string solvers (except OSTRICH). As a result, we do *not* generate the string constraints for the 12 instances in SEQEXT that contain matchAll_e . Let us use SEQEXT-M to denote the 12 instances in SEQEXT that contain matchAll_e and SEQEXT-N to denote the other 48 instances in SEQEXT.

Table 2. Experimental results on SEQEXT.

		cvc5	Z3	Z3-noodler	OSTRICH	OSTRICH ^{SEQ}
SEQEXT-N	sat	19	4	4	20	23
	unsat	21	6	4	18	25
	solved	40	10	8	38	48
	unknown/timeout	8	38	40	10	0
	avg. time (s)	10.83	0.13	0.05	15.92	3.53
SEQEXT-M	sat	-	-	-	-	12
	unsat	-	-	-	-	0
	solved	-	-	-	-	0
	unknown/timeout	-	-	-	-	0
	avg. time (s)	-	-	-	-	1.77

From Table 2, we can see that Z3 and Z3-noodler solve only 10 and 8 out of 48 constraints in SEQEXT-N respectively, and cvc5 and OSTRICH solve 40 and 38 instances respectively, indicating that the complexity of the string constraints generated from the SeqStr constraints. On the other hand, OSTRICH^{SEQ} solves all of them, demonstrating its superiority over solvers in solving the SeqStr constraints that contain the operations specialized to string sequences. For the solving time per instance, OSTRICH^{SEQ} spends 3.53 seconds on average, which is significantly lower than cvc5 and OSTRICH, spending 10.83 and 15.92 seconds on average respectively. For the constraints in SEQEXT-M, OSTRICH^{SEQ} successfully solves all 12 instances with an average time of 1.77 seconds. As mentioned before, for the constraints in SEQEXT-M, we do not generate string constraints or compare against cvc5, Z3, Z3-noodler and OSTRICH, since these constraints cannot be expressed using only the basic operations of string theory.

7 Related work

String constraint solving. String constraint solving has received considerable attentions in literature. Amadini [5] provides a comprehensive survey for the literature up to 2021. In particular, approaches to string constraint solving are classified into three main categories: automata-based (relying on finite state automata to represent the domain of string variables and to handle string operations, e.g., [3,30,17,28,36,12,22]), word-based (algebraic approaches based on

systems of word equations, e.g., [35,7,25,49,11,37,45,44,12,22]), and unfolding-based approaches (explicitly reducing each string into a number of contiguous elements denoting its characters, e.g., [41,33,40,34,6,24]).

From another perspective, there are two lines of work aiming for building practical string solvers: (1) one could support as many constraints as possible, but primarily resort to heuristics, offering no completeness/termination guarantee. Many solvers are usually implemented as string theory solvers inside SMT solvers, allowing combination with other theories, most commonly the theory of integers for string lengths. Some (non-exhaustive) examples include CVC4/5 [35,7], Z3 [25,13], Z3-str2/3/4 [49,11,37], S3(P) [45,44], Trau [2] (or its variants Trau+ [4]), Slent [46], etc. More recent ones include Z3str3RE [12] and Z3-Noodler [22] (which is based on stabilization-based algorithm [15,21] for solving word equations with regular constraints). (2) The second approach is to develop solvers for decidable fragments. Solvers in this category include Norn [3], SLOTH [30] and OSTRICH [17,20], which are usually based on complete decision procedures, (e.g. [28,4,36])

We also mention that there are solvers which emphasize certain aspects of string constraint solving by providing dedicated methods or optimization. These include, for instance, those for more expressive regular expressions [10,18], regex-counting [31], string/integer conversion and flat regular constraints [47], Not-Substring Constraint [1], integer data type [19], etc.

Sequences as an extension of strings. Strings and sequences are closely related, but may exhibit in different forms. For instance, a string can be considered as a sequence over a finite alphabet. Jez etc. [32] recently studied sequence theories which are an extension of theories of strings with an infinite alphabet of letters, together with a corresponding alphabet theory (e.g. linear integer arithmetic). They provide decision procedures based on parametric automata/transducers, giving rise to a sequence constraint solver *SeCo*. Our work is different in that we essentially work on a sequence theory where the element is instantiated by the string type.

Theory of arrays. There are many studies on solving formulas in the theory of arrays or its extensions, for instance, [43,16,29,26,23,27], which are also related to the sequence theory. Note that in these studies, the elements are considered to be either of a generic type or of integer type. In other words, theory of arrays where the elements are of the string type has not been defined and investigated therein.

8 Conclusion

In this paper, we have proposed *SeqStr*, a logic of string sequences, which supports a wealth of string and sequence operations—especially their interactions—commonly in real-word string-manipulation programs. We have provided a decision procedure for the straight-line fragment, which is implemented as a new tool *OSTRICH^{SEQ}*. The experiments on both hand-crafted and real-word benchmarks demonstrate that *OSTRICH^{SEQ}* is an effective and promising tool for testing, analysis and verification of string-manipulation programs in practice.

References

1. P. A. Abdulla, M. F. Atig, Y. Chen, B. P. Diep, L. Holík, D. Hu, W. Tsai, Z. Wu, and D. Yen. Solving not-substring constraint with flat abstraction. In H. Oh, editor, *Programming Languages and Systems - 19th Asian Symposium, APLAS 2021, Chicago, IL, USA, October 17-18, 2021, Proceedings*, volume 13008 of *Lecture Notes in Computer Science*, pages 305–320. Springer, 2021.
2. P. A. Abdulla, M. F. Atig, Y. Chen, B. P. Diep, L. Holík, A. Rezine, and P. Rümmer. Flatten and conquer: a framework for efficient analysis of string constraints. In *PLDI*, pages 602–617, 2017.
3. P. A. Abdulla, M. F. Atig, Y. Chen, L. Holík, A. Rezine, P. Rümmer, and J. Stenman. String constraints for verification. In *CAV*, pages 150–166, 2014.
4. P. A. Abdulla, M. F. Atig, B. P. Diep, L. Holík, and P. Janku. Chain-free string constraints. In *ATVA*, pages 277–293, 2019.
5. R. Amadini. A survey on string constraint solving. *ACM Comput. Surv.*, 55(2):16:1–16:38, 2023.
6. R. Amadini, G. Gange, and P. J. Stuckey. Propagating lex, find and replace with dashed strings. In W.-J. van Hove, editor, *Integration of Constraint Programming, Artificial Intelligence, and Operations Research*, pages 18–34, Cham, 2018. Springer International Publishing.
7. H. Barbosa, C. W. Barrett, M. Brain, G. Kremer, H. Lachnitt, M. Mann, A. Mohamed, M. Mohamed, A. Niemetz, A. Nötzli, A. Ozdemir, M. Preiner, A. Reynolds, Y. Sheng, C. Tinelli, and Y. Zohar. cvc5: A versatile and industrial-strength SMT solver. In D. Fisman and G. Rosu, editors, *Tools and Algorithms for the Construction and Analysis of Systems - 28th International Conference, TACAS 2022, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2022, Munich, Germany, April 2-7, 2022, Proceedings, Part I*, volume 13243 of *Lecture Notes in Computer Science*, pages 415–442. Springer, 2022.
8. C. Barrett, P. Fontaine, and C. Tinelli. The SMT-LIB Standard: Version 2.7. Technical report, Department of Computer Science, The University of Iowa, 2025. Available at www.SMT-LIB.org.
9. C. Barrett and C. Tinelli. *Satisfiability Modulo Theories*, pages 305–343. Springer International Publishing, Cham, 2018.
10. M. Berzish, J. D. Day, V. Ganesh, M. Kulczynski, F. Manea, F. Mora, and D. Nowotka. Towards more efficient methods for solving regular-expression heavy string constraints. *Theor. Comput. Sci.*, 943:50–72, 2023.
11. M. Berzish, V. Ganesh, and Y. Zheng. Z3str3: A string solver with theory-aware heuristics. In D. Stewart and G. Weissenbacher, editors, *2017 Formal Methods in Computer Aided Design, FMCAD 2017, Vienna, Austria, October 2-6, 2017*, pages 55–59. IEEE, 2017.
12. M. Berzish, M. Kulczynski, F. Mora, F. Manea, J. D. Day, D. Nowotka, and V. Ganesh. An SMT solver for regular expressions and linear arithmetic over string length. In A. Silva and K. R. M. Leino, editors, *Computer Aided Verification - 33rd International Conference, CAV 2021, Virtual Event, July 20-23, 2021, Proceedings, Part II*, volume 12760 of *Lecture Notes in Computer Science*, pages 289–312. Springer, 2021.
13. N. Bjørner, N. Tillmann, and A. Voronkov. Path feasibility analysis for string-manipulating programs. In *TACAS*, pages 307–321, 2009.
14. N. Bjørner, V. Ganesh, R. Michel, and M. Veanes. An smt-lib format for sequences and regular expressions. In *Proceedings of the 10th International Workshop on*

- Satisfiability Modulo Theories (SMT'12)*, Manchester, UK, June 30–July 1 2012. Affiliated with IJCAR 2012.
15. F. Blahoudek, Y. Chen, D. Chocholatý, V. Havlena, L. Holík, O. Lengál, and J. Síc. Word equations in synergy with regular constraints. In M. Chechik, J. Katoen, and M. Leucker, editors, *Formal Methods - 25th International Symposium, FM 2023, Lübeck, Germany, March 6-10, 2023, Proceedings*, volume 14000 of *Lecture Notes in Computer Science*, pages 403–423. Springer, 2023.
 16. A. R. Bradley, Z. Manna, and H. B. Sipma. What’s decidable about arrays? In E. A. Emerson and K. S. Namjoshi, editors, *Verification, Model Checking, and Abstract Interpretation, 7th International Conference, VMCAI 2006, Charleston, SC, USA, January 8-10, 2006, Proceedings*, volume 3855 of *Lecture Notes in Computer Science*, pages 427–442. Springer, 2006.
 17. T. Chen, Y. Chen, M. Hague, A. W. Lin, and Z. Wu. What is decidable about string constraints with the replaceall function. *PACMPL*, 2(POPL):3:1–3:29, 2018.
 18. T. Chen, A. Flores-Lamas, M. Hague, Z. Han, D. Hu, S. Kan, A. W. Lin, P. Rümmer, and Z. Wu. Solving string constraints with regex-dependent functions through transducers with priorities and variables. *Proc. ACM Program. Lang.*, 6(POPL):1–31, 2022.
 19. T. Chen, M. Hague, J. He, D. Hu, A. W. Lin, P. Rümmer, and Z. Wu. A decision procedure for path feasibility of string manipulating programs with integer data type. In D. V. Hung and O. Sokolsky, editors, *Automated Technology for Verification and Analysis - 18th International Symposium, ATVA 2020, Hanoi, Vietnam, October 19-23, 2020, Proceedings*, volume 12302 of *Lecture Notes in Computer Science*, pages 325–342. Springer, 2020.
 20. T. Chen, M. Hague, A. W. Lin, P. Rümmer, and Z. Wu. Decision procedures for path feasibility of string-manipulating programs with complex operations. *Proc. ACM Program. Lang.*, 3(POPL), 2019.
 21. Y. Chen, D. Chocholatý, V. Havlena, L. Holík, O. Lengál, and J. Síc. Solving string constraints with lengths by stabilization. *Proc. ACM Program. Lang.*, 7(OOPSLA2):2112–2141, 2023.
 22. Y. Chen, D. Chocholatý, V. Havlena, L. Holík, O. Lengál, and J. Síc. Z3-noodler: An automata-based string solver. In B. Finkbeiner and L. Kovács, editors, *Tools and Algorithms for the Construction and Analysis of Systems - 30th International Conference, TACAS 2024, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2024, Luxembourg City, Luxembourg, April 6-11, 2024, Proceedings, Part I*, volume 14570 of *Lecture Notes in Computer Science*, pages 24–33. Springer, 2024.
 23. P. Daga, T. A. Henzinger, and A. Kupriyanov. Array folds logic. In S. Chaudhuri and A. Farzan, editors, *Computer Aided Verification*, pages 230–248, Cham, 2016. Springer International Publishing.
 24. J. D. Day, T. Ehlers, M. Kulczynski, F. Manea, D. Nowotka, and D. B. Poulsen. On solving word equations using sat. In E. Filiot, R. Jungers, and I. Potapov, editors, *Reachability Problems*, pages 93–106, Cham, 2019. Springer International Publishing.
 25. L. de Moura and N. Bjørner. Z3: an efficient SMT solver. In *TACAS*, pages 337–340, 2008.
 26. S. Falke, F. Merz, and C. Sinz. Extending the theory of arrays: memset, memcpy, and beyond. In E. Cohen and A. Rybalchenko, editors, *Verified Software: Theories, Tools, Experiments - 5th International Conference, VSTTE 2013, Menlo Park, CA, USA, May 17-19, 2013, Revised Selected Papers*, volume 8164 of *Lecture Notes in Computer Science*, pages 108–128. Springer, 2013.

27. B. Farinier, R. David, S. Bardin, and M. Lemerre. Arrays made simpler: An efficient, scalable and thorough preprocessing. In *LPAR-22. 22nd International Conference on Logic for Programming, Artificial Intelligence and Reasoning, Awassa, Ethiopia, 16-21 November 2018*, volume 57 of *EPiC Series in Computing*, pages 363–380, 2018.
28. V. Ganesh, M. Minnes, A. Solar-Lezama, and M. C. Rinard. Word equations with length constraints: What’s decidable? In *HVC 2012*, pages 209–226, 2012.
29. P. Habermehl, R. Iosif, and T. Vojnar. What else is decidable about integer arrays? In R. M. Amadio, editor, *Foundations of Software Science and Computational Structures, 11th International Conference, FOSSACS 2008, Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2008, Budapest, Hungary, March 29 - April 6, 2008. Proceedings*, volume 4962 of *Lecture Notes in Computer Science*, pages 474–489. Springer, 2008.
30. L. Holík, P. Janku, A. W. Lin, P. Rümmer, and T. Vojnar. String constraints with concatenation and transducers solved efficiently. *PACMPL*, 2(POPL):4:1–4:32, 2018.
31. D. Hu and Z. Wu. String constraints with regex-counting and string-length solved more efficiently. In H. Hermanns, J. Sun, and L. Bu, editors, *Dependable Software Engineering. Theories, Tools, and Applications - 9th International Symposium, SETTA 2023, Nanjing, China, November 27-29, 2023, Proceedings*, volume 14464 of *Lecture Notes in Computer Science*, pages 1–20. Springer, 2023.
32. A. Jez, A. W. Lin, O. Markgraf, and P. Rümmer. Decision procedures for sequence theories. In *Proceedings of the 35th International Conference on Computer Aided Verification (CAV), Part II*, volume 13965, pages 18–40. Springer, 2023.
33. A. Kiezun, V. Ganesh, S. Artzi, P. J. Guo, P. Hooimeijer, and M. D. Ernst. Hampi: A solver for word equations over strings, regular expressions, and context-free grammars. *ACM Trans. Softw. Eng. Methodol.*, 21(4), Feb. 2013.
34. G. Li and I. Ghosh. Pass: String solving with parameterized array and interval automaton. In V. Bertacco and A. Legay, editors, *Hardware and Software: Verification and Testing*, pages 15–31, Cham, 2013. Springer International Publishing.
35. T. Liang, A. Reynolds, C. Tinelli, C. Barrett, and M. Deters. A DPLL(T) theory solver for a theory of strings and regular expressions. In *CAV*, pages 646–662, 2014.
36. A. W. Lin and P. Barceló. String solving with word equations and transducers: Towards a logic for analysing mutation XSS. In *Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL ’16*, pages 123–136, 2016.
37. F. Mora, M. Berzish, M. Kulczynski, D. Nowotka, and V. Ganesh. Z3str4: A multi-armed string solver. In M. Huisman, C. S. Pasareanu, and N. Zhan, editors, *Formal Methods - 24th International Symposium, FM 2021, Virtual Event, November 20-26, 2021, Proceedings*, volume 13047 of *Lecture Notes in Computer Science*, pages 389–406. Springer, 2021.
38. P. Rümmer. A constraint sequent calculus for first-order logic with linear integer arithmetic. In *Proceedings of the 15th International Conference on Logic for Programming, Artificial Intelligence and Reasoning*, volume 5330 of *LNCS*, pages 274–289. Springer, 2008.
39. P. Saxena, D. Akhawe, S. Hanna, F. Mao, S. McCamant, and D. Song. A symbolic execution framework for javascript. In *S&P*, pages 513–528, 2010.
40. P. Saxena, D. Akhawe, S. Hanna, F. Mao, S. McCamant, and D. Song. A symbolic execution framework for javascript. In *Proceedings of the 2010 IEEE Symposium on Security and Privacy, SP ’10*, page 513528, USA, 2010. IEEE Computer Society.

41. J. D. Scott, P. Flener, J. Pearson, and C. Schulte. Design and implementation of bounded-length sequence variables. In D. Salvagnin and M. Lombardi, editors, *Integration of AI and OR Techniques in Constraint Programming*, pages 51–67, Cham, 2017. Springer International Publishing.
42. Y. Sheng, A. Nötzli, A. Reynolds, Y. Zohar, D. Dill, W. Grieskamp, J. Park, S. Qadeer, C. Barrett, and C. Tinelli. Reasoning about vectors using an smt theory of sequences. In J. Blanchette, L. Kovács, and D. Pattinson, editors, *Automated Reasoning*, pages 125–143, Cham, 2022. Springer International Publishing.
43. A. Stump, C. W. Barrett, D. L. Dill, and J. R. Levitt. A decision procedure for an extensional theory of arrays. In *16th Annual IEEE Symposium on Logic in Computer Science, Boston, Massachusetts, USA, June 16-19, 2001, Proceedings*, pages 29–37. IEEE Computer Society, 2001.
44. M. Trinh, D. Chu, and J. Jaffar. S3: A symbolic string solver for vulnerability detection in web applications. In *CCS*, pages 1232–1243, 2014.
45. M. Trinh, D. Chu, and J. Jaffar. Progressive reasoning over recursively-defined strings. In *Computer Aided Verification - 28th International Conference, CAV 2016, Toronto, ON, Canada, July 17-23, 2016, Proceedings, Part I*, pages 218–240, 2016.
46. H.-E. Wang, S.-Y. Chen, F. Yu, and J.-H. R. Jiang. A symbolic model checking approach to the analysis of string and length constraints. In *ASE*, page 623633. ACM, 2018.
47. H. Wu, Y. Chen, Z. Wu, B. Xia, and N. Zhan. A decision procedure for string constraints with string/integer conversion and flat regular constraints. *Acta Informatica*, 61(1):23–52, 2024.
48. Z3Prover. Sequences | online z3 guide, 2025.
49. Y. Zheng, V. Ganesh, S. Subramanian, O. Tripp, M. Berzish, J. Dolby, and X. Zhang. Z3str2: an efficient solver for strings, regular expressions, and length constraints. *Formal Methods Syst. Des.*, 50(2-3):249–288, 2017.